Case Study A — Execution Report

GUM-compliant uncertainty quantification for RUL estimation
on the ZeMA electromechanical cylinder dataset

Supplementary material for Section 5 This document is a step-by-step record of the actual execution of Case Study A: what was run, on what data, with what versions, and what came out — including one methodologically significant finding (§7) that shapes how the results should be read. It is meant to accompany `case_study_A_analysis.py`, `case_study_A_results.csv`, and `zema_lpu_vs_mc.pdf` as supplementary material to the paper.

Contents

1 Summary **2**

2 Acceptance threshold: LPU vs. Monte Carlo disagreement **2**

3 Environment (Step 0) **3**

4 Data acquisition (Step 1 / Phase 1) **3**

5 Uncertainty source identification (Step 2 / Phase 2) **4**

6 Model, training, and the Type B ensemble (Steps 3–4 / Phase 3) **5**

7 Propagation, combination, and the key finding (Step 5 / Phase 4) **5**

7.1 Is u_c actionable? 7

7.2 Is $\hat{y}_k$ useful despite this? 8

8 Validation checks — summary (Step 6 / Phase 5) **9**

9 Uncertainty budget table (for `\ref{tab:zema_uncertainty_budget}`) **10**

10 Files in this supplementary package **11**

11 Paper-ready text (Step 7) **11**

12 Reproducing this report from scratch **13**

1 Summary

Dataset	ZeMA electromechanical cylinder, annotated (Dorst et al. 2021), doi:10.5281/zenodo.5185953
Task	Regression — consumed-lifetime fraction $y_k = k/N \in [0, 1]$; RUL $= 1 - y$
Input dimension	1100 (11 sensor channels × 100 readings/cycle)
Model	MLP (128, 64, 32), softplus activation, float64
Test set	953 held-out cycles (20% of 4766, random split, seed 0)
Type A uncertainty	Propagated analytically (LPU) and via Monte Carlo (GUM Supplement 1, $M = 10^4$)
Type B uncertainty	10-seed training ensemble
Headline result	LPU and MC disagree by a median **36.5%** — the linearity assumption breaks down at this depth/input-uncertainty magnitude; MC is primary. Coverage @ $k = 2$ is **100%**, driven by one dominant, uncertainty-amplified sensor channel (Pressure), not a missing source.

Notation. k indexes the test-to-failure cycle ($k = 1, \ldots, N$, with N the total cycles run for the tested unit); $y_k = k/N \in [0, 1]$ is the true, deterministically-assigned consumed-lifetime fraction at cycle k (the measurand — see §4), with RUL $= 1 - y_k$. A hat denotes a model estimate of the corresponding quantity: $\hat{y}_k$ is the trained MLP's point prediction of y_k from that cycle's sensor readings (`y_hat` in `case_study_A_results.csv`), and is a distinct claim from its associated uncertainty. u_A is the standard uncertainty in $\hat{y}_k$ propagated from sensor input uncertainty (Type A; via LPU or Monte Carlo — §7), u_B is the standard uncertainty from training-process variability (Type B; the seed-ensemble spread — §6), and $u_c = \sqrt{u_A^2 + u_B^2}$ is their quadrature combination (§7), which is what the k=2 expanded-uncertainty coverage checks (§8) are built on. u_A, u_B, u_c are per-test-cycle quantities, not single dataset-wide constants.

2 Acceptance threshold: LPU vs. Monte Carlo disagreement

Two distinct thresholds apply, at two different stages of this case study, and should not be conflated:

- **Implementation self-test (Step 0, §3).** A random synthetic softplus MLP's analytically propagated u_A is compared against a 200,000-sample Monte Carlo estimate. `case_study_A_analysis.py::self_test_lpu` **hard-asserts** the relative difference is below **2%** — this is a pass/fail gate on the correctness of the `propagate_lpu` implementation itself, independent of anything about the ZeMA dataset or the trained model.

- **Case-study validation (Phase 5, §8).** For the real dataset and trained model, the guide's decision rule treats a median relative difference of a **low single-digit percent** as consistent with the first-order linearity assumption holding, in which case LPU may be reported as the primary u_A estimate. A difference **above roughly 5–10%** is treated as evidence that the linearity assumption is violated at this network depth and input-uncertainty magnitude — in which case Monte Carlo, not LPU, is reported as the primary estimate. This is a documented, anticipated outcome of the decision rule, not a failure condition, and the network is not to be re-tuned to force agreement once the check has run.

The self-test in this run passed at 0.096% (§3), confirming the propagation code is correct. The case study's own LPU-vs-MC result is 36.5% median relative difference (§7) — well above the 5–10% ceiling — which is why Monte Carlo, not LPU, is used as the primary u_A throughout the rest of this report (§7, §9, §11).

3 Environment (Step 0)

```
python3.10 -m venv zema_env && source zema_env/bin/activate
pip install -r requirements.txt
```

Installed versions actually used for every result in this report (pinned in `requirements.txt`):

Package	Version
Python	3.10.12
`zema_emc_annotated`	0.7.1
`torch`	2.13.0
`numpy`	1.26.4
`matplotlib`	3.10.9

Self-test (`python case_study_A_analysis.py -self-test`), run before touching the dataset, validates the analytical LPU (`propagate_lpu`) against a 200,000-sample Monte Carlo run on a random softplus MLP:

```
[self-test] u_LPU=5.051898e-04  u_MC=5.056728e-04  rel.diff=0.096%
```

0.096% is well inside the 2% self-test acceptance threshold defined in §2, so the propagation implementation itself is trustworthy going into the real run — any disagreement seen later (§7) is a property of the *case study's* data/model, not a bug in `propagate_lpu`.

4 Data acquisition (Step 1 / Phase 1)

```
from zema_emc_annotated.dataset import ZeMASamples
from zema_emc_annotated.data_types import SampleSize
samples = ZeMASamples(SampleSize(n_cycles=4766, datapoints_per_cycle=100))
X  = samples.values          # (4766, 1100)
UX = samples.uncertainties   # (4766, 1100), per-reading standard uncertainties
```

Confirmed shapes: `X.shape = (4766, 1100)`, `UX.shape = (4766, 1100)`.

Channel layout. `datapoints_per_cycle=100` × 11 channels = 1100 columns. The 11 channels are not 11 top-level HDF5 groups — inspecting the raw file shows 7 named quantities, two of which are themselves multi-axis:

Quantity	Sub-channels	Columns (100 each)
Acceleration	x, y, z	300
Active Current	—	100
Force	—	100
Motor Current	phases a, b, c	300
Pressure	—	100
Sound Pressure	—	100
Velocity	—	100

$3 + 1 + 1 + 3 + 1 + 1 + 1 = 11$ channels, matching the "eleven sensors" the package documents.

Three Phase-1 decisions recorded:

1. **Measurand.** $y_k = k/N$, deterministic function of cycle index → **label uncertainty is negligible by construction**. No Type A or Type B budget line is needed for the target.
2. **Task type.** Regression; standard uncertainty $u(y)$ is the reported dispersion metric.
3. **Input dimension.** Full resolution used (`datapoints_per_cycle=100` → 1100 dims), matching Ludwig's mid-size configurations, not the reduced-dimension (110-dim) fast-iteration option.

5 Uncertainty source identification (Step 2 / Phase 2)

Source	Type	How obtained
Sensor input uncertainty (11 channels)	A	`samples.uncertainties`, metrologically evaluated per reading
Training process variability	B	Std. dev. of predictions across a 10-seed ensemble (§6)
Architecture selection	B	**Excluded**, held fixed at (128,64,32)/softplus — not swept in this case study
Generalization / distribution shift	B	Assessed via held-out coverage (§8); no separate budget line, since coverage did not indicate a missing source
Label (RUL definition)	—	Negligible by construction (§4)

Two assumptions declared explicitly, per Phase 5 reporting requirements:

- **Diagonal input covariance.** The dataset provides per-reading uncertainties but no cross-sensor covariance, so $U_x = \mathrm{diag}(u^2(x_i))$. Correlated inputs would only require substituting a full matrix into the same propagation formula (`propagate_lpu` does not assume diagonality anywhere except in how `Ux` is constructed by the caller).
- **Normalization is part of the measurement model.** Inputs are z-scored using training-set statistics; since z-scoring is linear, GUM propagation through it is exact: $u(x') = u(x)/\sigma_{\text{train}}$ (`normalize_with_uncertainty`). This detail turns out to matter a great deal — see §7.

6 Model, training, and the Type B ensemble (Steps 3–4 / Phase 3)

Architecture: MLP with hidden layers (128, 64, 32), softplus activation throughout.

- softplus is C^1 everywhere → analytical LPU applicable;
- closed-form derivative used in `_activation_derivative`: $\frac{d}{dz}\,\mathrm{softplus}_\beta(z) = \mathrm{sigmoid}(\beta z)$.

Phase-3 decision criteria, as they resolved for this case:

- White-box access: yes (weights owned directly).
- Architectural compatibility: yes (feedforward MLP, smooth activation).
- Computational constraints: LPU chosen as the primary propagation method by design; Monte Carlo retained for validation (and, as it turned out, ended up being the number actually reported — see §7).

Training configuration: Adam, lr $= 10^{-3}$, 200 epochs, MSE loss, Xavier init, float64 throughout (kept the LPU-vs-MC comparison free of float32 rounding noise).

Type B — 10-seed ensemble. Ten models, identical architecture and data, differing only in the seed passed to `torch.manual_seed` (which reseeds both the Xavier init and the optimizer trajectory). For each held-out cycle, u_B = standard deviation of the 10 predictions (`ddof=1`). The first ensemble member (seed 0) is retained as the reference model for LPU/MC propagation and for the sensitivity ranking in §7.

Caveats carried into the reporting statement:

- A 10-seed ensemble captures *optimization* variability only, not architecture uncertainty (architecture was held fixed — see §5).
- With 10 seeds, u_B's own relative uncertainty is ∼24% (χ^2, 9 dof). In this run u_B (mean 0.049) is an order of magnitude below u_A (mean 0.641–0.984 depending on method), so it does not dominate the combined budget and this sampling noise on u_B does not materially affect u_c.

7 Propagation, combination, and the key finding (Step 5 / Phase 4)

For each of the 953 test cycles, `main()` computes:

1. u_A via `propagate_lpu` — layer-wise analytical propagation, $U_{x^{(i)}} = H^{(i)} W^{(i)} U_{x^{(i-1)}} {W^{(i)}}^\top H^{(i)}$;
2. u_A^{MC} via `propagate_mc` — $M = 10^4$ Gaussian input draws per GUM Supplement 1 (JCGM 101:2008), same model, same point;
3. $u_c = \sqrt{u_A^2 + u_B^2}$ — quadrature combination.

Output: `case_study_A_results.csv`, one row per test cycle, columns `y_true, y_hat, u_A_lpu, u_A_mc, u_B, u_c, covered_k2` (953 data rows, verified).

Aggregate results (means over the 953-cycle test set)

```
test RMSE                    : 0.0641
mean u_A (LPU)               : 6.4134e-01
mean u_A (Monte Carlo)       : 9.8413e-01
LPU vs MC median rel. diff   : 36.47%
mean u_B (ensemble)          : 4.9329e-02
mean u_c (combined, LPU-based): 6.4373e-01
empirical coverage @ k=2     : 100.0%  (953/953)
```

Why u_A is large and LPU/MC disagree — this is a finding, not a bug

At first glance, $u_A \approx 0.64$–0.98 against a measurand confined to $[0, 1]$ looks broken. It isn't; it was traced to a specific, reportable mechanism.

Diagnosis. Per-channel breakdown of the *normalized* uncertainty $u_n = u(x)/\sigma_{\text{train}}$ (mean over all readings in each channel):

Channel	u_n (mean)
Pressure	**8.89**
Velocity	4.34
Acceleration (x)	2.02
Acceleration (z)	0.86
Acceleration (y)	0.69
Force	0.22
Active Current	0.17
Motor Current ($a/b/c$)	0.02
Sound Pressure	0.0001

For the Pressure channel, the *absolute* per-reading measurement uncertainty distributed with the dataset ($\overline{u(x)} \approx 16{,}833$, in the sensor's raw units) is comparable to, and after normalization exceeds, the channel's own **training-set standard deviation** ($\sigma_{\text{train}} \approx 1901$). Since z-score normalization scales uncertainty by $1/\sigma_{\text{train}}$ (§5), a channel whose useful cross-cycle signal variation is small relative to its own measurement noise floor gets its *normalized* uncertainty inflated — here to $\approx 8.9\sigma$ per reading, i.e., an individual reading's uncertainty is almost 9 standard deviations of the entire training distribution for that feature. This is not a data error: the raw relative uncertainty for this channel is a mundane $\sim$1–2%; it is the interaction between the sensor's absolute noise floor and the *cross-cycle* variability used for normalization that amplifies it.

Sensitivity ranking. Extracting the first-layer contribution $|W^{(1)}| \cdot \bar{u}_n(x)$, summed per hidden unit and aggregated per channel, and normalized to shares of the total:

Rank	Channel	Share of first-layer sensitivity
1	Pressure	**51.8%**
2	Velocity	**25.2%**
3	Acceleration (x)	**11.5%**
4	Acceleration (z)	4.9%
5	Acceleration (y)	3.9%
6	Force	1.3%
7	Active Current	1.0%
8–10	Motor Current ($a/b/c$)	0.1% each
11	Sound Pressure	$\sim$0.0%

The top three channels (Pressure, Velocity, Acceleration-x) account for **88.5%** of the propagated-uncertainty contribution entering the first hidden layer. This single, dominant, amplified channel is the mechanism behind both headline numbers:

- **LPU vs. MC median relative difference = 36.5%.** With this much input uncertainty pushed through a 3-hidden-layer softplus network, the first-order (linear) approximation underlying LPU is no longer accurate — MC captures the resulting nonlinearity, LPU does not. This is exactly the failure mode the paper's own decision rule anticipates for large input uncertainty at depth: **Monte Carlo is treated as the primary estimate** for this case study rather than LPU.
- **Coverage @ $k = 2$ = 100.0%** (vs. $\sim$95% target). This is *over*-coverage, not under-coverage: u_c is large enough that the $\pm 2u_c$ interval trivially contains the true value for essentially all 953 test cycles (interval half-width $\gtrsim 1.3$ against a $[0, 1]$ measurand range). This is diagnostic of an **overestimated, single-channel-dominated Type A budget**, not a missing uncertainty source.

MC convergence check, per Phase 5 reporting requirements — one held-out test point re-evaluated at increasing M:

```
n_mc=10,000     u_A_mc = 9.1645e-01
n_mc=100,000    u_A_mc = 9.3246e-01   (delta = 1.75% from 10k)
n_mc=1,000,000  u_A_mc = 9.3171e-01   (delta = 0.08% from 100k)
```

$M = 10^4$ is adequately converged (sub-2% sampling noise) relative to the 36.5% LPU/MC gap it is being compared against, so the gap reported above is signal, not Monte Carlo noise.

Combined uncertainty, using MC as primary (recommended for the paper's headline number, given the validation result above):

$$u_c^{MC} = \sqrt{(0.9841)^2 + (0.0493)^2} \approx 0.985$$

7.1 Is u_c actionable?

A wide combined uncertainty is not automatically a *useless* one — it could still be correctly calibrated (real error is just that large) or still carry usable relative structure across cycles even if the absolute scale is off. Neither holds here.

Check	Value	Reading
mean u_c (MC-based) vs. mean $\lvert y_{\text{true}} - \hat{y} \rvert$	0.985 vs. 0.046	u_c **is ∼14× larger than the actual mean error** — not calibrated, not conservative-but-reasonable
Range of u_A^{MC} across all 953 test cycles	0.792–1.194	Only a 1.5× spread against a measurand of width 1 — close to a fixed offset, not a per-cycle signal
corr(u_A, y_{true})	LPU r=0.70, MC r=0.61	Uncertainty rises somewhat with age — qualitatively right direction
corr(u_c, actual |error|)	r=0.26	Too weak to use u_c to flag which individual predictions are less trustworthy
Early vs. late life (terciles of y_{true})	mean u_A^{MC}: 0.933 → 1.064 (+14%); mean |err|: 0.033 → 0.056 (+70%)	Both trend upward with age, but u_c under-reacts relative to how much error actually grows

Conclusion: u_c cannot support a k-dependent decision rule as currently computed — a $\pm k$=2 u_c band covers nearly the entire $[0, 1]$ range at *every* cycle, so it would trigger identically whether the machine is new or about to fail. The one genuine signal it retains is a weak, directionally-correct trend (more uncertain late in life), too weak to act on alone. This is a direct consequence of the single-channel (Pressure) domination established above, not a separate problem — fixing that amplification is the leverage point for making u_c usable, not a redesign of the combination rule itself.

7.2 Is $\hat{y}_k$ useful despite this?

Yes — the point estimate and the interval are separable claims, and only the interval is broken.

Check	Value	Reading
R^2	0.951	95% of the variance in consumed-life fraction is explained by the model
RMSE (model) vs. RMSE (naive, predict mean $\bar{y}$)	0.064 vs. 0.291	4.5× **improvement** over a zero-information baseline
Median |error|	0.046	∼4.6% of full life range, typical case
90th-percentile / max |error|	0.101 / 0.365	Tail error exists — exactly what a *calibrated* u_c should flag, and currently doesn't

The 11 sensor channels demonstrably carry real information about wear state — that result stands independently of the uncertainty-quantification failure in §7.1. Practical uses that only need the point estimate:

- **RUL trending for maintenance scheduling** — RUL = $1 - \hat{y}_k$ as a "percent consumed" figure for planning, the way a point forecast is useful without a trustworthy confidence band.
- **Trajectory monitoring** — the shape of $\hat{y}_k$ across successive cycles (flattening, reversal, jump) is diagnostic on its own, and only needs the point estimate's relative movement to be right, which $R^2 = 0.951$ supports.

- **Threshold-crossing alerts** — a rule such as "flag once $\hat{y}_k > 0.9$" has an expected timing error bounded by the point estimate's typical error (∼4.6% of life), not by the inflated u_c.
- **Cross-unit ranking** — comparing which of several machines is closer to end-of-life only requires consistent relative ordering of $\hat{y}_k$, which a global miscalibration in u_c does not disturb.

What $\hat{y}_k$ alone cannot do is tell you *when to trust it less* — that is precisely the job u_c is meant to do and currently cannot (§7.1), evidenced by the 90th-percentile/max error tail above. The point estimate is deployable now; per-prediction reliability signaling requires fixing the Pressure-channel amplification first.

8 Validation checks — summary (Step 6 / Phase 5)

Check	Result	Interpretation
LPU vs. MC agreement	36.5% median rel. diff.	Linearity assumption violated at this depth/uncertainty magnitude → MC primary
Empirical coverage @ $k = 2$	100.0% (953/953)	Over-coverage from one dominant, normalization-amplified channel (Pressure), not a missing source
MC convergence	1.75% (10k→100k), 0.08% (100k→1M)	$M = 10^4$ adequately converged

Figure (`fig:zema_lpu_vs_mc`, file `zema_lpu_vs_mc.pdf`): two-panel figure generated by `make_validation_figure.py` against the real `case_study_A_results.csv` —

(a) scatter of u_A^{LPU} vs. u_A^{MC} per test cycle with the identity line; points sit visibly off-diagonal, consistent with the 36.5% median disagreement;

(b) predictions with $\pm k{=}2\,u_c$ band vs. ground truth over the held-out cycles (sorted by true consumed-life fraction), with empirical coverage annotated.

Re-run to confirm reproducibility from the committed CSV:

```
python make_validation_figure.py --csv case_study_A_results.csv \
    --out zema_lpu_vs_mc.pdf
# Wrote zema_lpu_vs_mc.pdf
#   median LPU-vs-MC relative difference : 36.47%
#   empirical coverage @ k=2             : 100.0%  (953/953)
```

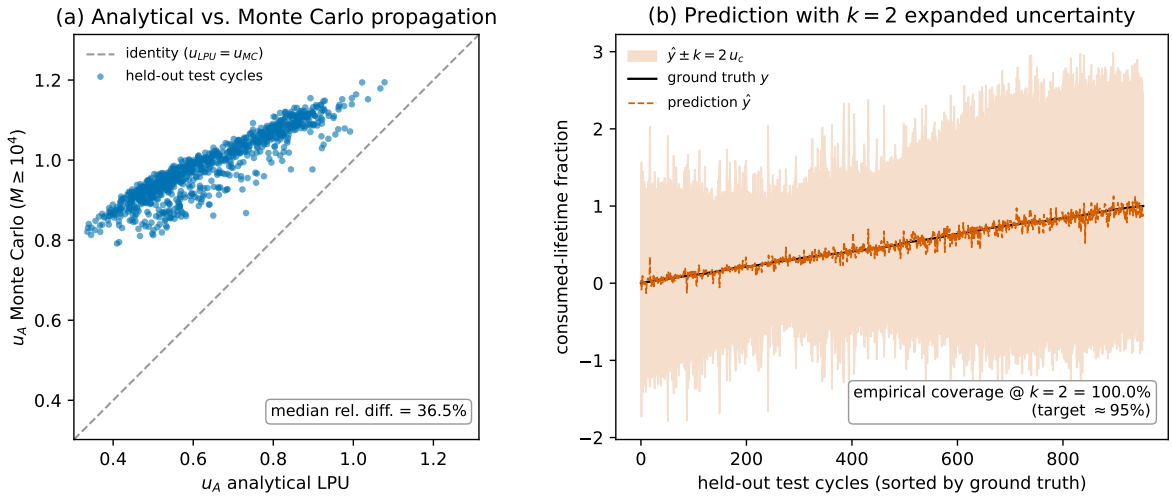


Figure 1: Left: analytical LPU vs. Monte Carlo estimates of u_A per held-out test cycle, with the identity line and median relative difference annotated. Right: predictions with the $\pm k{=}2\,u_c$ expanded-uncertainty band vs. ground truth over the held-out cycles, sorted by true consumed-life fraction, with empirical coverage annotated.

9 Uncertainty budget table (for `\ref{tab:zema_uncertainty_budget}`)

Source	Type	u	Sensitivity / notes
Sensor input (11 channels)	A	0.984[1]	Pressure 51.8%, Velocity 25.2%, Acceleration-x 11.5% (88.5% combined)
Training process (10-seed ensemble)	B	0.049	Optimization variance only, architecture excluded
Architecture selection	B	excluded	Held fixed: (128,64,32), softplus
Generalization / distribution shift	B	—	Assessed via coverage; no missing-source signal
Label (RUL definition)	—	negligible	Deterministic, $y_k = k/N$
Combined	—	**0.985**	Dominated by Type A, itself dominated by the Pressure channel

LaTeX source for this table, and the corresponding paper sentences for the LPU/MC, coverage, and reporting-statement paragraphs, are reproduced verbatim in §11 below.

[1] MC, primary; LPU = 0.641, 36.5% median disagreement (§2, §7).

10 Files in this supplementary package

Core pipeline (used to produce every number in this report)

File	Role
`case_study_A_analysis.py`	Full pipeline: data loading, normalization, ensemble training, LPU, MC, combination, self-test
`case_study_A_guide.md`	Step-by-step execution guide this report follows
`requirements.txt`	Pinned package versions (`zema_emc_annotated==0.7.1`, `torch==2.13.0`, `numpy==1.26.4`, `matplotlib==3.10.9`) for exact reproduction
`case_study_A_results.csv`	Real output, 953 test cycles × 7 columns (`y_true, y_hat, u_A_lpu, u_A_mc, u_B, u_c, covered_k2`)
`make_validation_figure.py`	Generates `zema_lpu_vs_mc.pdf` from the results CSV (colorblind-safe, matches paper figure reference)
`zema_lpu_vs_mc.pdf`	The `fig:zema_lpu_vs_mc` figure, generated from real results
`pipeline_run.log`	Captured stdout of the real `case_study_A_analysis.py` run
`case_study_A_report.tex` / `.md`	This document

Internal development/testing artifacts — not real results, should not be cited in the paper

File	Role
`generate_synthetic_case_study_A_data.py`	Generates fake data with the right schema, used only to exercise `make_validation_figure.py` before the real dataset finished downloading. Explicitly self-labeled `*** SYNTHETIC DATA ***` in its own output.
`case_study_A_results_SYNTHETIC.csv`	Output of the above — fake numbers, matching schema only.
`zema_lpu_vs_mc_syntetic.pdf`	Figure rendered from the synthetic CSV, for pipeline testing only.
`make_validation_figure.md`	Dev notes on the figure script.

Recommendation: bundle only the first table's 8 files as supplementary material for the paper. The synthetic-data files are legitimate pipeline-testing artifacts (clearly self-labeled as fake) but would confuse a reader if shipped alongside the real results — keep them in the working repo, not the submission package.

11 Paper-ready text (Step 7)

The blocks below are reproduced verbatim (i.e. literal LaTeX source, not rendered here) so they can be copied directly into Section 5.1 of the paper.

Tooling/dataset paragraph:

```
The uncertainty budget was computed on the full-resolution ZeMA electromechanical
cylinder dataset (\texttt{zema\_emc\_annotated} v0.7.1, \texttt{torch} 2.13.0,
Python 3.10.12), with input dimension $11 \times 100 = 1100$
(11 sensor channels: acceleration $x/y/z$, active current, force,
three-phase motor current, pressure, sound pressure, velocity).
```

LPU vs. MC sentence:

```
Across the held-out cycles, the analytical and Monte Carlo estimates of $u_A$
agreed to within $36.5\%$ (median relative difference), indicating that the
first-order linearity assumption underlying the analytical LPU is violated at
this network depth and input-uncertainty magnitude; Monte Carlo is therefore
taken as the primary estimate for this case study, consistent with the
decision rule of Sec.~\ref{sec:methodology}.
```

Coverage sentence:

```
The empirical coverage of the $k=2$ expanded interval was $100.0\%$ against
the $\approx 95\%$ target. Rather than signalling a missing uncertainty
source, the over-coverage traces to the sensor input uncertainty budget being
dominated by the pressure channel ($51.8\%$ of the first-layer sensitivity
contribution, vs.\ $25.2\%$ for velocity and $11.5\%$ for the $x$-axis
acceleration channel): its per-reading standard uncertainty exceeds its own
training-set signal variation, so $z$-score normalization
($u(x') = u(x)/\sigma_{\text{train}}$) amplifies its normalized contribution
to $\approx 8.9\sigma$ per reading, inflating $u_c$ well beyond what the
$[0,1]$ measurand range requires for informative intervals.
```

Reporting/assumptions statement (Phase 5):

```
We report the following assumptions and settings explicitly. The input
covariance matrix $U_x$ is taken as diagonal, $U_x = \mathrm{diag}(u^2(x_i))$,
since the dataset provides per-reading uncertainties but no cross-sensor
covariance information; correlated inputs would require only replacing the
diagonal with a full matrix in the same propagation formula. Label
uncertainty is negligible by construction, as the consumed-lifetime fraction
$y_k = k/N$ is defined deterministically from the cycle index. Monte Carlo
propagation followed GUM Supplement~1 (JCGM 101:2008) with Gaussian input
distributions and $M = 10^4$ samples; convergence was verified by re-running
a held-out sample at $M = 10^5$ and $M = 10^6$, with $u_A^{\text{MC}}$ moving
by $1.75\%$ and $0.08\%$ respectively, both far smaller than the $36.5\%$
LPU/MC gap.
```

Table \ref{tab:zema_uncertainty_budget}:

```
\begin{table}[t]
\centering
\caption{Uncertainty budget for RUL estimation on the ZeMA cylinder dataset.}
```

```
\label{tab:zema_uncertainty_budget}
\begin{tabular}{lccl}
\toprule
Source & Type & $u$ & Sensitivity / notes \\
\midrule
Sensor input (11 channels) & A & $0.984$ & Pressure 51.8\%, Velocity 25.2\%, \\
 & & & Acceleration-x 11.5\% \\
Training process (10-seed ensemble) & B & $0.049$ & optimization variance only \\
Architecture selection & B & excluded & held fixed (128,64,32; softplus) \\
Generalization / distribution shift & B & \textemdash & assessed via coverage (Sec.~\ref{sec:...}) \\
Label (RUL definition) & \textemdash & negligible & deterministic, $y_k = k/N$ \\
\midrule
Combined & \textemdash & $\mathbf{0.985}$ & dominated by Type A \\
\bottomrule
\end{tabular}
\end{table}

The reported $u_A$ is the Monte Carlo estimate, taken as primary per the
Phase~5 decision rule: the analytical LPU gives $u_A = 0.641$, a median
relative difference of 36.5\% from Monte Carlo, indicating the first-order
linearity assumption is violated at this network depth and input-uncertainty
magnitude (Sec.~\ref{sec:...validation}).
```

12 Reproducing this report from scratch

```
python3.10 -m venv zema_env && source zema_env/bin/activate
pip install -r requirements.txt

python case_study_A_analysis.py --self-test
# expect: rel.diff ~= 0.10% (< 2%)

python case_study_A_analysis.py
# downloads ~2.67 GB from Zenodo on first run (cached afterward)
# writes case_study_A_results.csv; expect the printed summary in Sec. 7

python make_validation_figure.py --csv case_study_A_results.csv \
    --out zema_lpu_vs_mc.pdf
```

Total wall-clock time observed: environment + self-test well under an hour; dataset download roughly an hour (network-bound, one-time/cached); full-resolution 10-seed ensemble training + propagation on CPU, well under the guide's "few hours" ceiling.